

★ WEEK 8 · PART 2 · PRACTICE

The Kalman Filter, Worked by Hand

Run the Full Recursion; Track a Drifting Hedge Ratio

Predict and update, step by step, on real financial problems

Part 2's engine, with the arithmetic shown. We run the predict-update recursion over four steps and watch the gain settle, then track a time-varying hedge ratio - the pairs thread - with one Kalman update by hand.

2P

PART 2 · APPLICATIONS

The Recursion, Computed by Hand

Case A: run the Kalman recursion over four prints, watch the gain reach a steady state, and see smoothing beat filtering. Case B: track a dynamic hedge ratio with one update. Then a Claude Code lab.

What Part 2 Told Us

Now we run it - with numbers

1

Three inference problems

Filtering estimates the state now, smoothing the state then with hindsight, forecasting the state ahead.

2

The predict-update cycle

Predict the state forward, then correct it with each new observation by the Kalman gain. We run four steps by hand.

3

Parameters can be hidden states

A hedge ratio that drifts is just another state to filter. We update one by hand - the pairs thread in state space.

Everything here is a calculator exercise: four prints, one recursion, one return pair.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from the recursion or the two equations.
- Run predict-update by hand, step by step.
- Then read what the filtered state tells you.

What you should watch

- How the gain settles to a steady state.
- How smoothing improves on filtering.
- How a hedge ratio drifts and gets tracked.

Run the Full Recursion

From one step to a real-time filter

Part 1 did one filtering step; now we run the recursion. Using the same local level model - a fair value hidden behind noisy prints - we predict and update across four prints, tracking the estimate and its variance, and watch the Kalman gain settle to a steady state.

The recursion

- Predict the state and its variance forward.
- Correct it with each print by the gain.
- Carry the variance to the next step.

What we'll do by hand

- Run four predict-update steps.
- Watch the gain converge near 0.33.
- See the estimate track the fair value.

The Predict-Update Cycle

Four formulas, run every step

$$\begin{aligned}\text{predict: } R &= C + \sigma_{\eta}^2 \\ \text{gain: } K &= \frac{R}{R + \sigma_{\varepsilon}^2} \\ \text{update mean: } m &\leftarrow m + K (P - m) \\ \text{update var: } C &\leftarrow (1 - K) R\end{aligned}$$

Each step: grow the variance (predict), form the gain, correct the mean toward the print, and shrink the variance (update). Then repeat.

Four Steps of the Recursion

Predict and update, print by print

t	print P	R (predict)	gain K	m (filtered)	C (var)
1	100.6	0.140	0.359	100.215	0.090
2	100.2	0.130	0.342	100.210	0.085
3	100.9	0.125	0.334	100.440	0.084
4	100.5	0.124	0.331	100.460	0.083

The gain falls from 0.359 toward a steady 0.33, and the variance settles - the filter reaches an equilibrium as data accumulates.

One Full Step, in Detail

Step 3, worked end to end

$$\begin{aligned}\text{step 3: } R &= 0.0854 + 0.04 = 0.125, & K &= \frac{0.125}{0.125 + 0.25} = 0.334 \\ m &= 100.21 + 0.334(100.9 - 100.21) = \mathbf{100.44}, & C &= (1 - 0.334) 0.125 = 0.084\end{aligned}$$

What happened

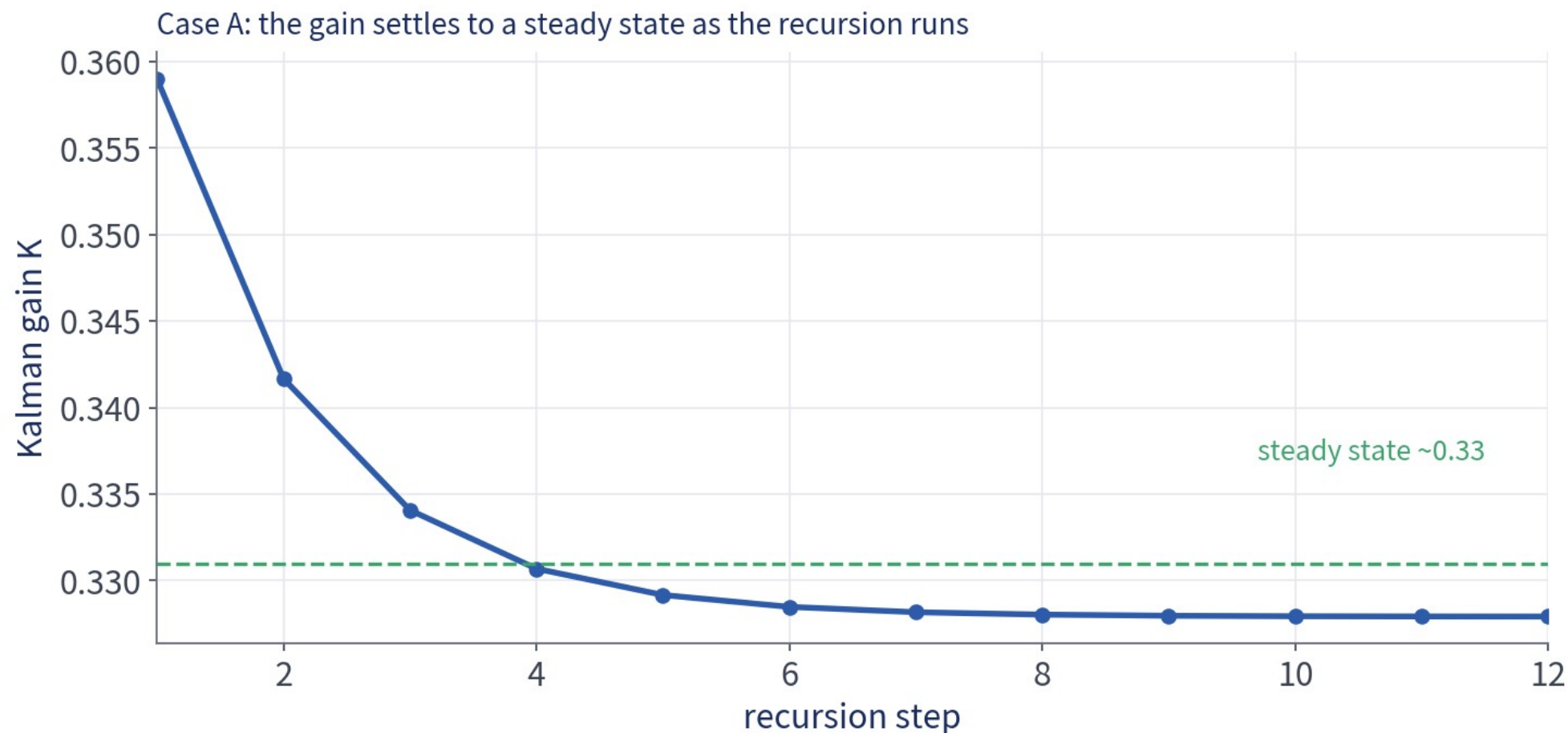
- The print 100.9 was above the estimate.
- The gain 0.334 pulled the mean up to 100.44.
- The variance shrank to 0.084.

Why it settles

- Predict grows the variance by 0.04.
- Update shrinks it by the gain.
- The two balance - a steady-state gain.

The Gain Settles

Run the recursion longer and the Kalman gain converges to a steady state near 0.33. After that, every print is trusted by the same fixed fraction - the filter has learned the right balance of signal and noise for this model



Filtering vs Smoothing

Real time, or with hindsight

Filtering (real time)

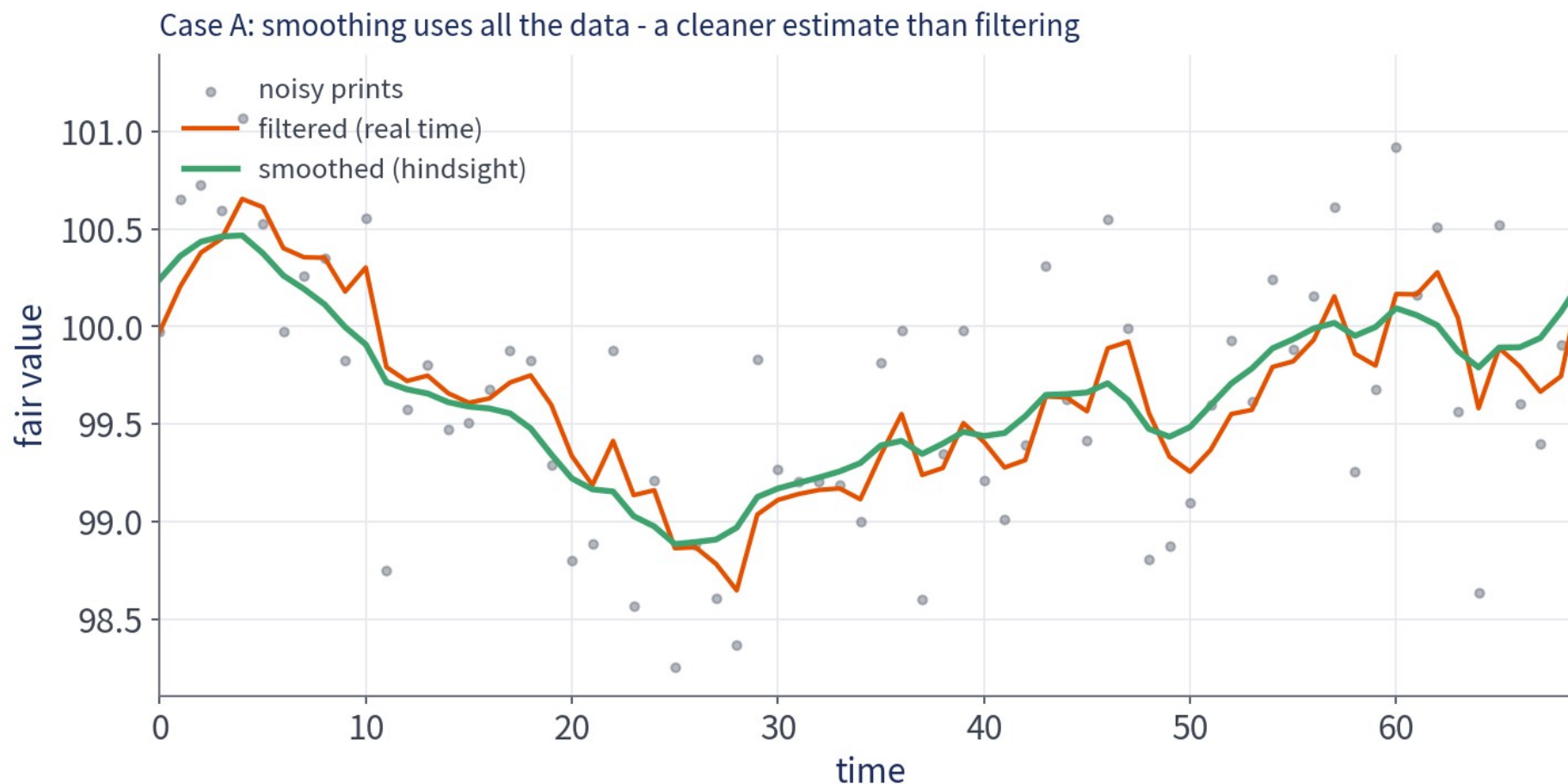
- Uses only data up to now.
- The best you can do live, at each step.
- A little jumpy - it can't see ahead.

Smoothing (hindsight)

- Uses the whole series to revise each point.
- A cleaner estimate of what the state was.
- Only available after the fact - for research.

Smoothing Beats Filtering, After the Fact

The filtered estimate (orange) uses only the past, the smoothed estimate (green) uses the whole series to revise each point, so it is cleaner and less jumpy. Use filtering to trade live, smoothing to study what really happened



What the Recursion Buys You

A living estimate, not a static average

The filter gives you

- A fair value updated every tick.
- A variance - how sure you are right now.
- A gain that adapts to the noise level.

The choice of tool

- Filter when you must act in real time.
- Smooth when you're analyzing history.
- Same model, two information sets.

A Hedge Ratio That Drifts

The pairs thread enters state space

In pairs trading you hedge stock A with a multiple of stock B - the hedge ratio. A fixed ratio from one regression goes stale. Treat the hedge ratio as a hidden state that drifts, and the Kalman filter updates it as new returns arrive. We do one update by hand.

The setup

- Regress A's return on B's return.
- The slope is the hedge ratio, beta.
- Let beta drift slowly over time.

What we'll do by hand

- Write beta as a hidden state.
- Take one new return pair.
- Update beta with one Kalman step.

The Hedge Ratio as a Hidden State

A drifting regression slope

$$\begin{aligned} \text{observation: } r_{A,t} &= \beta_t r_{B,t} + \varepsilon_t && \text{(regress A on B)} \\ \text{state: } \beta_t &= \beta_{t-1} + \eta_t && \text{(the hedge ratio drifts)} \end{aligned}$$

The observation

- A's return equals beta times B's, plus noise.
- The regressor is B's return - it changes daily.
- So the gain changes each step too.

The state

- Beta takes a small random step each day.
- It drifts, but slowly.
- That is the hidden thing we track.

One Update of the Hedge Ratio

From a single return pair

$$\text{predict var: } P = 0.040 + 0.001 = 0.041$$

$$\text{innovation: } e = r_A - \beta r_B = 1.2 - 0.80(1.0) = 0.40$$

$$\text{gain: } K = \frac{P r_B}{r_B^2 P + \sigma_\varepsilon^2} = \frac{0.041}{0.041 + 0.20} = 0.17$$

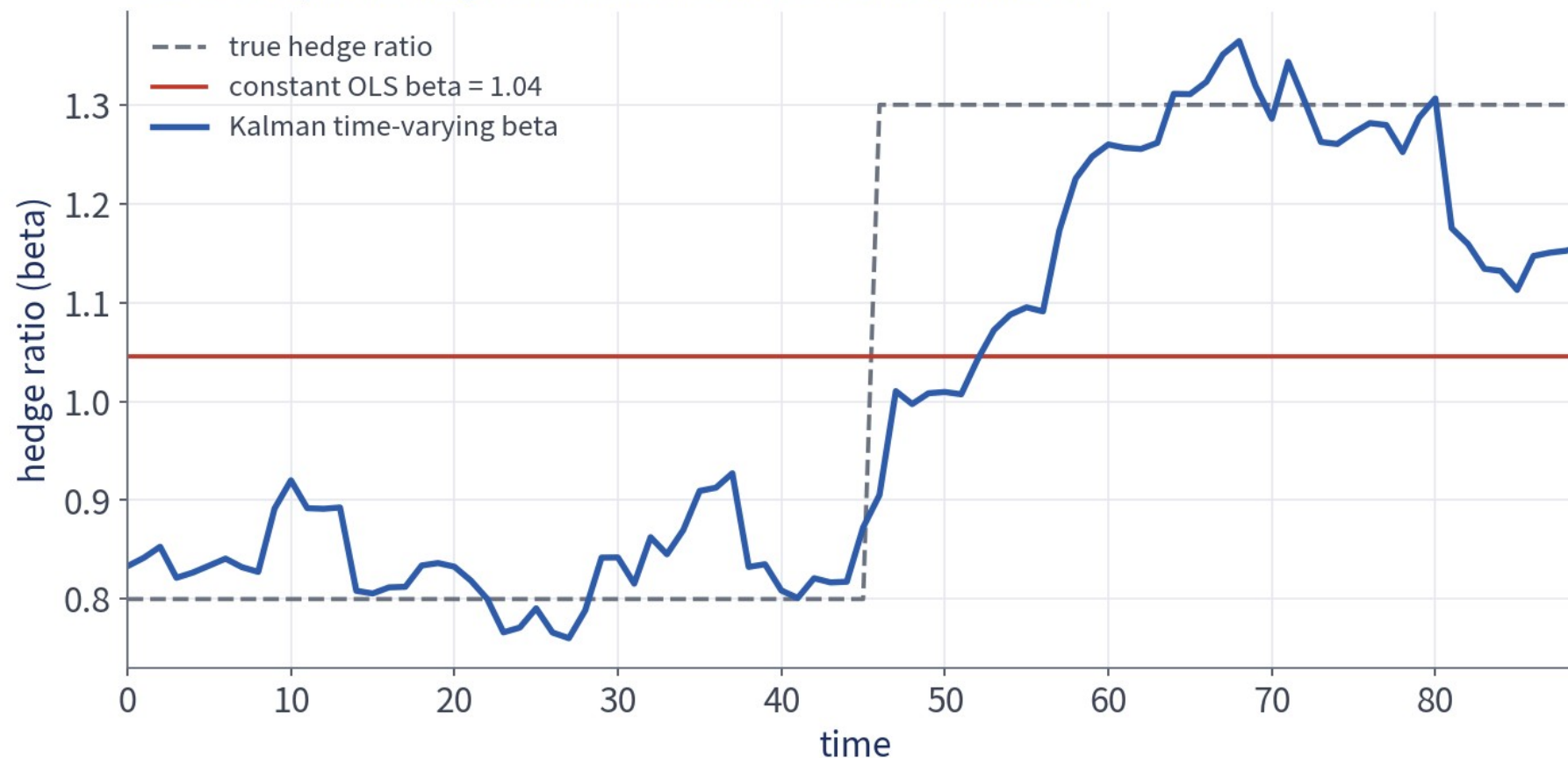
$$\text{update: } \beta \leftarrow 0.80 + 0.17(0.40) = \mathbf{0.87}$$

A's 1.2% return beat 0.80 x B's 1.0%, an innovation of 0.40. The gain 0.17 nudges beta from 0.80 up to 0.87 - a gradual, sensible revision.

The Dynamic Hedge Ratio Tracks the Shift

When the true hedge ratio jumps, the Kalman estimate (blue) follows it within a few steps, while a single constant OLS beta (red) sits stuck in between - hedging badly both before and after. A drifting state beats a fixed number

Case B: the dynamic hedge ratio tracks the shift; constant OLS misses it



Why a Dynamic Hedge Wins

The pairs trade, kept in tune

A constant hedge

- Comes from one regression on old data.
- Goes stale when the relationship shifts.
- Leaves you over- or under-hedged.

A filtered hedge

- Updates the ratio as new returns arrive.
- Adapts to a regime change automatically.
- Keeps the spread properly market-neutral.

The Common Pattern

One recursion, many states

Both cases run the same engine. Whatever the hidden state - a fair value or a hedge ratio - the filter predicts it forward, then corrects it by the gain when data arrives. The gain does the thinking: it weighs your prior against the new observation, automatically.

The recipe

- Predict the state and its variance.
- Form the gain from those variances.
- Correct the state, shrink the variance.

Why it's powerful

- The same loop tracks any hidden state.
- The gain adapts to the noise, step by step.
- Filtering is live; smoothing is hindsight.

Reproduce the Recursion, Then Scale Up

Direct Claude Code to check your arithmetic

GOAL Verify the recursion and the hedge-ratio update, then run both on real data.

[INPUT] Prompt · Claude Code

Write and run `part2_practice.py` that:

CASE A - the recursion:

- * run the predict-update loop over the four prints;
- * print R , K , m , C at each step; show K settling;
- * add an RTS smoother and compare to the filter.

CASE B - dynamic hedge ratio:

- * from beta 0.80 (var 0.04) and returns $r_B=1.0$, $r_A=1.2$, compute the innovation, gain, and new beta;
- * then track beta on a real pair vs a constant OLS.

Match every printed number to the slides.

Check against the slides

- gain settles near 0.33.
- step 3: $m = 100.44$.
- innovation $e = 0.40$.
- beta: 0.80 $\rightarrow$ 0.87.

Deliverable: `part2_practice.py` whose printed numbers match the recursion and the hedge-ratio update.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What are the two halves of each recursion step?

**Predict, then update
by the gain**

Q2 How does smoothing beat filtering?

**It uses the whole
series, with hindsight**

Q3 Why filter the hedge ratio instead of fixing it?

**It drifts - a filtered
beta adapts**

All three come from the predict-update cycle, the filter-vs-smoother chart, and the dynamic hedge ratio.

Three Pitfalls to Avoid

Where the recursion slips

Common mistakes

- "Forget to carry the variance forward."
- "Use smoothing to trade in real time."
- "Fit the hedge ratio once and leave it."

The reality

- The variance is the memory - carry it every step.
- Smoothing peeks ahead; only filter live.
- Let the hedge ratio drift and be re-filtered.

Two Calculations to Remember

Part 2 applications in one screen

1

Run the recursion, watch the gain settle

Over four prints the gain fell from 0.359 toward a steady 0.33 and the variance settled. Predict grows the variance, update shrinks it, and the two balance into a steady-state filter.

2

Smoothing beats filtering, after the fact

Filtering uses only the past and is the best you can do live; smoothing uses the whole series to revise each point and is cleaner. Filter to trade, smooth to study.

3

Track a drifting hedge ratio

Treating beta as a hidden state, one return pair ($r_B=1.0$, $r_A=1.2$) nudged it from 0.80 to 0.87 by a gain of 0.17. A filtered hedge adapts to regime shifts a constant OLS beta misses.

Next — Part 3 practice: the midterm synthesis, tying filtering and smoothing into the pairs strategy.